



AULA 5 UNIDADE 5

Integração do Desenvolvimento ao Git



Introdução

- O trabalho de desenvolvedor de *software* requer constante cooperação com outros profissionais.
- Muitas vezes, as equipes são fisicamente separadas, o que exige código-fonte compatível.
- A colaboração é essencial para garantir que todos os desenvolvedores estejam trabalhando em harmonia, além de promover uma compreensão mais profunda do projeto.
- A troca de ideias pode levar a soluções mais criativas e inovadoras.
- A colaboração ainda ajuda a garantir que o projeto seja entregue dentro do cronograma e do orçamento.



Versionamento com *GitFlow*

- O desenvolvimento de software envolve pessoas, tecnologias e processos.
- O Git é uma ferramenta de versionamento que permite uma gestão mais eficiente e organizada do código-fonte.
- GitFlow:
 - É um modelo de *branching* que organiza o fluxo de trabalho.
 - Permite a separação do desenvolvimento de novas funcionalidades, correção de erros e a criação de diferentes versões do código-fonte para ambientes de desenvolvimento, homologação e produção.
- É um fluxo de trabalho simples projetado para ser facilmente compreendido pela equipe de desenvolvimento,

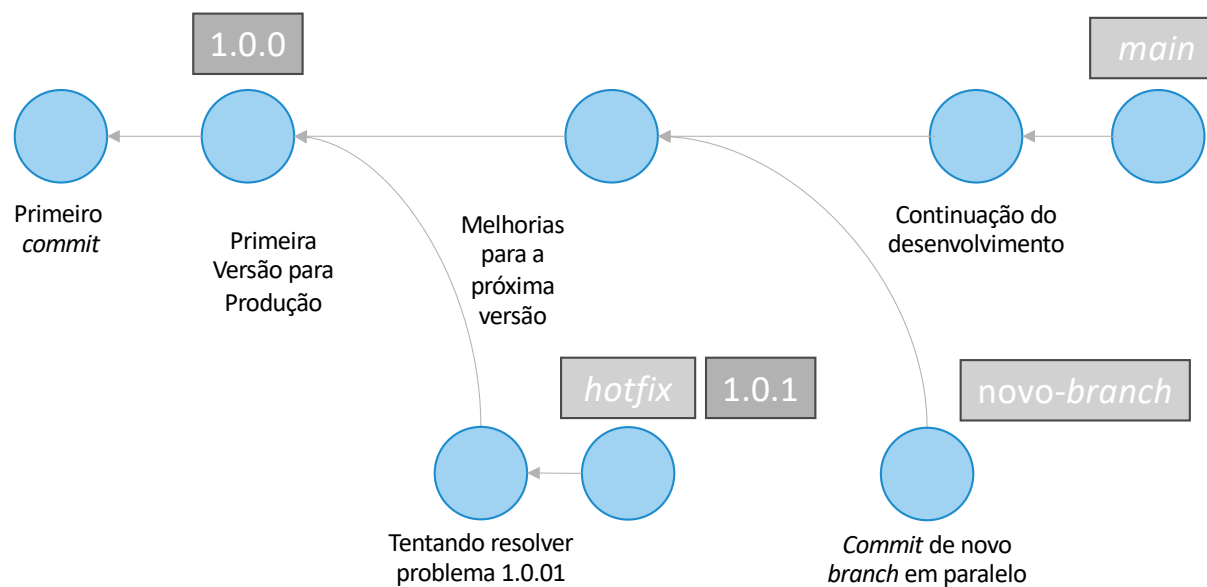


Versionamento com *GitFlow*

- Integração contínua é um processo em que os códigos desenvolvidos pelos desenvolvedores são testados e validados automaticamente antes de serem disponibilizados para uso.
- Liberações de código diárias para evitar grandes atualizações que possam causar impactos significativos na aplicação.
- Conceitos:
 - Linha principal, *trunk* ou *master*: código de produção.
 - Ramificação (*branching*): linhas independentes de desenvolvimento.
 - Marcação (*tagging*): rótulo da versão do código.
- Ao concluir um desenvolvimento realizado em um ramo (branch), um desenvolvedor precisará integrá-lo ao restante do código.



Versionamento com *GitFlow*



Versionamento com *GitFlow*

Branches					
-	<i>main</i>	<i>hotfix</i>	<i>release</i>	<i>develop</i>	<i>feature</i>
Repositório	<i>origin</i>	<i>local</i>	<i>origin</i>	<i>origin</i>	<i>local</i>
O que contém?	Código em Produção ou <i>trunk</i>	Correção de <i>bug</i>	Irà para produção	Irà para <i>release</i>	Funcionalidade futura
Padrão de nome	-	hotfix-*	release-*	-	Livre
Tempo de vida	Infinito	Até ser corrigido	Até estar pronto para produção	Infinito	Enquanto é implementada
Branch de origem	-	<i>main</i>	<i>develop</i>	<i>main</i>	<i>develop</i>
Branch de destino	-	<i>main, develop e release</i>	<i>main e develop</i>	<i>feature e release</i>	<i>develop</i>
Exemplo de <i>tag</i>	1.1.1.4	1.1.15	1.2	-	-
Observação	Cada <i>merge</i> gera uma nova versão	<i>Merge</i> com <i>main</i> , depois em <i>develop</i>	<i>Merge</i> com <i>main</i> , depois em <i>develop</i>	-	Uma <i>branch</i> para cada funcionalidade



Versionamento de Código em Equipe

- Os *branches* fundamentais são *main* e *develop*:
 - *main* reside no repositório central (*origin*), contendo código já liberado para produção.
 - Sua *tag* (rótulo) sempre é alterada após correção de *bug* em *hotfix* (exemplo, 1.1.14 -> 1.1.15) ou quando uma nova *release* fica disponibilizada (exemplo, 1.1.15 -> 1.2).
 - *develop* é um *branch* de *main* e incorpora as correções de *hotfix* e as novas funcionalidades de *feature* para no futuro ser incorporado no *release*.
 - *develop* é limpo ao ser liberado para o *release* e não receberá novas funcionalidades até *release* ser incluído em *main*.
- Importante observar que *hotfix* e *feature* na verdade são vários *branches*, um para cada correção de *bug* ou funcionalidade específica, sendo que devem ser nomeados de forma a refletir o *bug* a ser corrigido ou a funcionalidade a ser implementada.
- Também há um *branch* para cada *release*.



Versionamento de Código em Equipe

- `$ git checkout -b develop main` cria o *branch develop* a partir do *main*.
- `$ git checkout main` muda para o *branch main*.
- `$ git merge --no-ff release-1.1` faz a fusão (*merge*) da *release* número 1.1 com o *main*.
- `$ git tag -a v1.1 -m "Versão 1.1"` rotula o *branch main* com o número de versão 1.1.



Resolvendo Conflitos

- Boas práticas para trabalho em equipe no GitFlow:
 - Evitar espaços em branco desnecessários, como linhas vazias e erros de tabulação. O comando `git diff --check` antes de um `commit` verificar a presença desse tipo de erro.
 - Garantir que um `commit` tenha uma alteração independente, ou seja, uma única correção de *bug* ou funcionalidade nova.
 - Faça uma mensagem de `commit` descritiva, incluindo a motivação da alteração.
- As boas práticas ajudam no trabalho da equipe, mas conflitos de versionamento no *merge* de *branchs* é caso comum na integração contínua.



Resolvendo Conflitos

- Considere que *usuario1* e *usuario2* estão trabalhando de forma independente em uma nova implementação.
- A implementação consiste de dois novos arquivos e tem o nome *feature-15*.
- Os usuários clonam o repositório central nas suas máquinas locais, incluindo o *branch develop*.
- Vamos considerar que ambos usuários vão criar um novo *branch* e fazer alterações em arquivos diferentes.



Resolvendo Conflitos

usuario1	usuario2
<pre>\$ git clone REMOTO . \$ git branch feature-15 \$ git checkout feature-15 \$ nano arquivo2.c \$ git add arquivo2.c \$ git commit -a -m "feature-15 – adiciona arquivo2.c"</pre>	<pre>\$ git clone REMOTO . \$ git branch feature-15 \$ git checkout feature-15 \$ nano arquivo3.c \$ git add arquivo3.c \$ git commit -a -m "feature-15 – adiciona arquivo3.c"</pre>
	<pre>\$ git checkout develop \$ git merge feature-15 \$ git push origin develop</pre>
<pre>\$ git checkout develop \$ git merge feature-15 \$ git push origin develop Aqui será gerado conflito!!! \$ git fetch origin \$ git merge origin/develop \$ git push origin develop</pre>	



Conclusão

- O conjunto de boas práticas do GitFlow ajudam na colaboração da equipe.
- Organizar os *branches* de acordo com uma nomenclatura precisa e significativa reduz a necessidade de comunicação fora do controle de versão.
- É natural que cada organização desenvolva novas boas práticas com o decorrer dos projetos.
- Não podemos esquecer dos princípios de *feedback* e aprendizado contínuo no Fluxo de Valor de TI.



GRADUAÇÃO ONLINE **FGV**
TECNOLÓGICA